

SQL Query Linter & Style Fixer

Objective

- > To automatically format, clean, and structurally refactor SQL queries by standardizing identifier casing, enforcing keyword capitalization, detecting costly anti-patterns, and converting legacy implicit comma-joins and nested subqueries into modern SQL syntax.

Problem Statement

- > Development teams frequently encounter poorly formatted, inconsistent, and legacy SQL queries containing camelCase column names, PascalCase tables, and unstandardized keywords. Furthermore, implicit comma-joins and deeply nested subqueries lead to high maintenance complexity and degraded performance. Manual refactoring is highly error-prone and labor-intensive, while plain LLM-based refactoring can introduce syntax errors and hallucinations.

Technologies Used

- > Python
- > Language Refactoring SDK (Google GenAI SDK - gemini-2.5-flash)
- > SQL AST Parsing Library (sqlglot)
- > File Handling (SQL File Scanner & In-Memory Uploader)
- > User Interface (Streamlit Framework)

System Architecture

- > Read raw SQL queries from the sandbox input, uploaded files, or folders.
- > Sanitize and strip markdown formatting code blocks and bold/italic indicators.
- > Standardize casing of keywords and columns (casing to snake_case, keywords to uppercase).
- > Detect anti-patterns and log diagnostic warnings (such as explicit SELECT *).
- > Perform AST-based refactoring of joins and CTE extractions using Gemini LLM.
- > Run local validation loops to self-correct any generated syntax anomalies.
- > Output the clean, formatted SQL and generate a colored diff comparison view.

Modules

> Sanitization Module

- Removes markdown block fences and formatting asterisks/underscores without corrupting database identifiers.

> Phase 1 Linter Module

- Executes keyword capitalization, identifier conversions, and logs SELECT * warnings locally at zero cost.

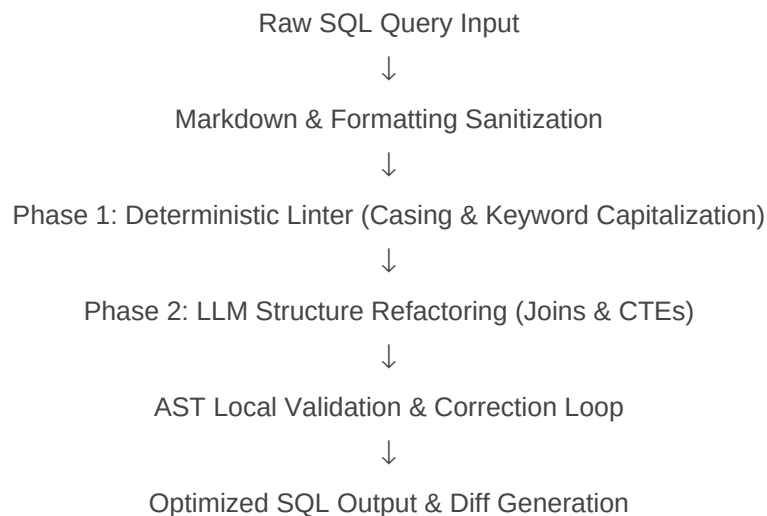
> Phase 2 Refactoring Module

- Engages the Gemini 2.5 LLM with few-shot instructions to translate legacy comma-joins and extract subqueries into CTEs.

> Verification & Self-Correction Module

- Validates generated output against a local SQL parser and loops back compilation errors to the LLM to self-heal.

Workflow



Advantages

- > Reduces manual refactoring and formatting overhead.
- > Guarantees syntactic correctness through local verification loop testing.
- > Prevents billing inflation by diagnosing SELECT * anti-patterns.
- > Offers dual interfaces (Streamlit dashboard and CLI mode) to fit different workflows.

Future Enhancements

- > Configurable target SQL dialect translations (Postgres, MySQL, Oracle, BigQuery).
- > Integration into CI/CD git pre-commit pipeline hooks.
- > Schema-aware validation matching table metadata catalogs.
- > AI-based database index recommendations based on query filters.

Conclusion

> The SQL Query Linter & Style Fixer offers a comprehensive, secure, and developer-friendly platform that automates legibility optimization and query structuring. By linking rule-based local formatting parser scripts with advanced self-correcting LLM validation loops, it guarantees clean styling and syntactically sound database queries.